Helios

High-Frequency Trading Matching Engine

Technical Design & Performance Report

Submitted by:

Saksham Madan
24124040

Navneet Kashyap
24124032

April 1, 2026

Contents

1 Introduction **2**
1.1 Scope and Goals . . . 2

2 System Architecture **2**
2.1 High-Level Overview . . . 2
2.2 Layer Decomposition . . . 3

3 Core Design Pillars **3**
3.1 Pillar 1 — Lock-Free Concurrency . . . 3
3.1.1 The SPSC Ring Buffer . . . 3
3.1.2 Memory Ordering Contract . . . 3
3.1.3 False Sharing Elimination . . . 4
3.2 Pillar 2 — User-Space Memory Management . . . 4
3.2.1 The Memory Pool . . . 4
3.2.2 The Intrusive List . . . 5
3.3 Pillar 3 — Cache-Local Data Structures . . . 5
3.3.1 The Limit Order Book . . . 5
3.3.2 Order Indexing . . . 5
3.4 Pillar 4 — Kernel Bypass and CPU Isolation . . . 6
3.4.1 Thread Affinity . . . 6
3.4.2 Real-Time Scheduling . . . 6
3.4.3 The Spin-Wait Barrier . . . 6

4 Matching Engine Logic **6**
4.1 Order Lifecycle . . . 6
4.2 Price-Time Priority (FIFO) . . . 7
4.3 Iterator Safety During Erasure . . . 7

5 Hardware Telemetry **8**
5.1 TSC Calibration . . . 8
5.2 Measurement Strategy . . . 8

6 Performance Results **8**
6.1 Throughput . . . 8
6.2 Latency Distribution . . . 9
6.3 Key Observations . . . 10

7 Project Structure **10**
7.1 Build System . . . 10

8 Design Trade-offs and Limitations **11**

9 Future Work **11**

10 Conclusion **11**

A Benchmark Environment **12**

1 Introduction

Modern electronic markets execute millions of trades per second. At the heart of every exchange sits a **Limit Order Book (LOB)** — the data structure that maintains all outstanding buy and sell orders and matches them when their prices overlap. The software that drives this process, the *matching engine*, is one of the most latency-sensitive programs in existence. A delay of a single microsecond can be the difference between a profitable trade and a missed opportunity.

Helios is a ground-up C++20 implementation of a LOB matching engine designed to explore the following question: *how close to bare hardware can a software matching engine operate without a custom kernel or FPGA?*

Helios is able to arrive at a median per-order processing latency of **38–48 ns** and sustained throughput of approximately **20–21 million orders per second** on commodity x86-64 hardware, achieved entirely in user space through four architectural pillars described in this report.

1.1 Scope and Goals

- Design a two-thread, lock-free producer/consumer pipeline.
- Eliminate all heap allocations from the steady-state hot path.
- Isolate the matching thread from OS scheduling interference.
- Measure true per-order latency using hardware cycle counters.

2 System Architecture

2.1 High-Level Overview

Helios decomposes the matching problem into two concurrent threads that communicate exclusively through a lock-free ring buffer. No mutex, semaphore, or condition variable is used at any point on the critical path.

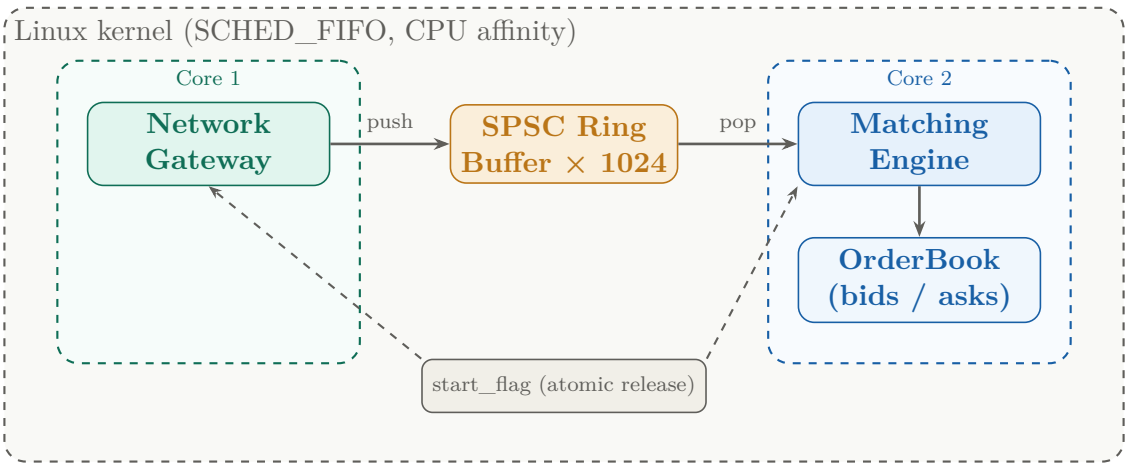

Figure 1: High-level system architecture. Two threads on isolated cores communicate through a single lock-free SPSC queue. The `start_flag` atomic ensures simultaneous launch after all OS configuration is complete.

2.2 Layer Decomposition

The codebase is organised into four horizontal layers, each with a single responsibility.

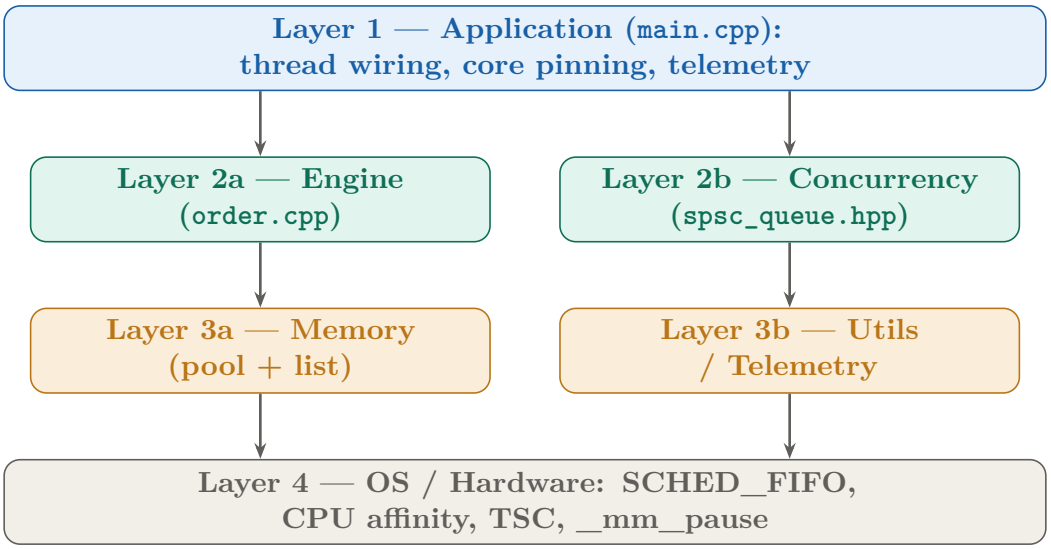

Figure 2: Four-layer architecture. Each layer depends only on the layers below it.

3 Core Design Pillars

3.1 Pillar 1 — Lock-Free Concurrency

3.1.1 The SPSC Ring Buffer

The gateway and engine threads communicate through a Single-Producer Single-Consumer (SPSC) ring buffer of capacity 1024 slots. The key insight is that with exactly one writer and one reader, no lock is ever needed — only careful use of atomic memory orderings.

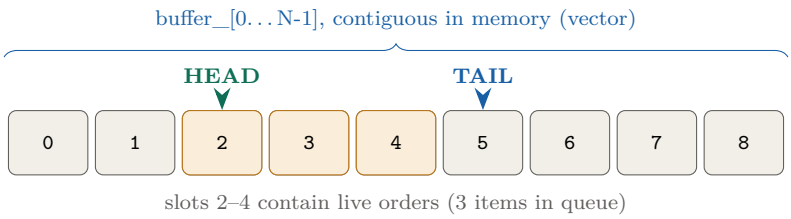

Figure 3: SPSC ring buffer state. HEAD (consumer-owned) and TAIL (producer-owned) never contend on the same atomic variable.

3.1.2 Memory Ordering Contract

The correctness of the entire queue rests on two pairs of atomic operations:

Operation	Ordering	Guarantee
`tail_.load` (producer)	`relaxed`	Safe — only producer writes `tail`; no sync needed
`head_.load` (producer)	`acquire`	Sees consumer's latest `head` write; validates full-check
`buffer[tail] = item`	—	Data write; must complete before tail is published
`tail_.store` (producer)	`release`	Publishes data write to consumer atomically
`head_.load` (consumer)	`relaxed`	Safe — only consumer writes `head`
`tail_.load` (consumer)	`acquire`	Synchronises with producer's `release`; sees fresh data
`out = move(buffer[head])`	—	Data read; guaranteed visible after acquire
`head_.store` (consumer)	`release`	Frees slot; producer sees it via next acquire on head

Table 1: Acquire/release pairing in the SPSC queue. Each `release` store is paired with an `acquire` load on the other thread, forming a happens-before relationship without a full memory fence.

3.1.3 False Sharing Elimination

`head_` and `tail_` are each wrapped in a struct declared `alignas(64)`, forcing them onto separate 64-byte cache lines. Without this, every producer write to `tail_` would invalidate the consumer's cache line containing `head_` , causing unnecessary cache coherence traffic and reducing throughput.

3.2 Pillar 2 — User-Space Memory Management

3.2.1 The Memory Pool

Every resting `Order` object is allocated from a custom arena pre-allocated at startup. `operator new` and `operator delete` are overloaded directly on the `Order` struct, making all call sites use pool allocation transparently.

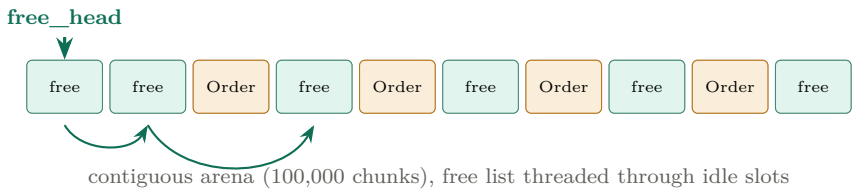

Figure 4: Memory pool layout. Free chunks form a singly-linked list through their own raw bytes (union trick). Allocation and deallocation are O(1) pointer operations.

The pool stores up to 100,000 `Order` objects. The union trick is critical: when a chunk

is free, its bytes hold a `Chunk*` pointer to the next free slot; when live, those same bytes hold a fully-constructed `Order`. A `static_assert` enforces `sizeof(Order) >= sizeof(Chunk*)`, preventing silent memory corruption if the type ever shrinks below pointer size.

3.2.2 The Intrusive List

At each price level, orders are queued in FIFO order using an intrusive doubly-linked list. *Intrusive* means the `prev` and `next` link pointers live inside the `Order` struct itself, not in a separately allocated wrapper node. This eliminates all allocations from the enqueue path and enables O(1) cancellation: given only the `Order*`, the list can splice the node out without any traversal.

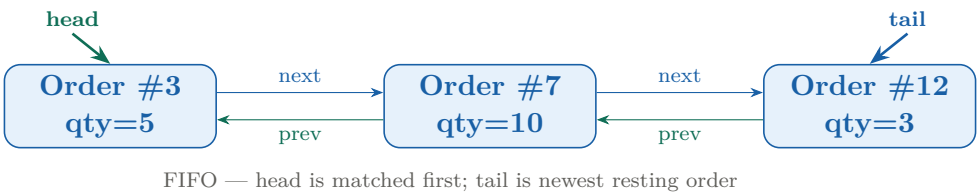

Figure 5: Intrusive doubly-linked list at a single price level. `prev`/`next` pointers reside inside the `Order` struct, requiring zero separate allocations.

3.3 Pillar 3 — Cache-Local Data Structures

3.3.1 The Limit Order Book

The LOB maintains two sorted vectors of price levels — `bids` (descending) and `asks` (ascending). Using `std::vector` instead of a tree structure (e.g. `std::map`) keeps all price levels contiguous in memory, enabling the CPU's prefetcher to load them ahead of use.

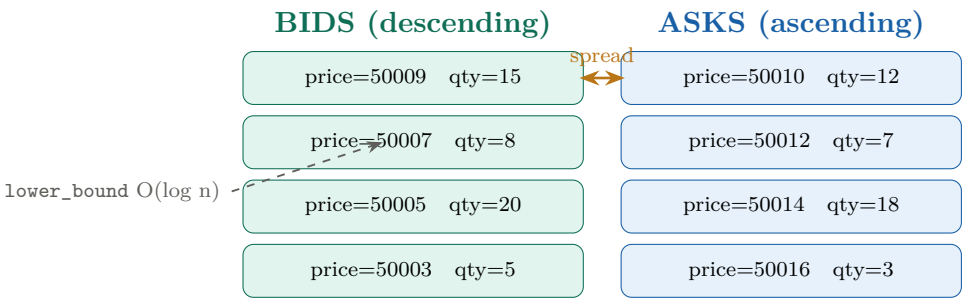

Figure 6: Limit order book. Each side is a sorted `std::vector<PriceLevel>`. Binary search (`std::lower_bound`) locates a price in O(log n), and the vector's contiguous layout ensures excellent cache utilisation.

3.3.2 Order Indexing

A `std::unordered_map<u64, Order*>` maps every live order ID directly to its pool pointer. This provides O(1) cancel-by-ID: the engine receives a cancel request, looks up the pointer, reads `price` and `side` from the `Order` itself, binary-searches the correct side's vector, and splices the node from the intrusive list — all without scanning a single price level.

3.4 Pillar 4 — Kernel Bypass and CPU Isolation

3.4.1 Thread Affinity

`pthread_setaffinity_np` pins the gateway thread to Core 1 and the engine thread to Core 2. Core 0 is reserved for the Linux OS. This eliminates the latency jitter caused by the kernel rescheduling either thread onto a different physical core mid-run.

3.4.2 Real-Time Scheduling

`pthread_setschedparam` elevates the engine thread to the `SCHED_FIFO` policy with maximum priority. Under `SCHED_FIFO`, the Linux kernel will not preempt this thread for any normal-priority task. This requires the binary to run with `sudo`.

3.4.3 The Spin-Wait Barrier

Both threads spin-wait on a shared `std::atomic<bool> start_flag` after spawning. `main()` sets the flag only after thread pinning and RT scheduling are confirmed. This ensures that hardware configuration is complete and both threads are waiting on *their assigned cores* before any order is ever pushed. The `memory_order_release` store pairs with the threads' `memory_order_acquire` loads, guaranteeing that all prior initialisation work is visible to both threads the instant the flag flips.

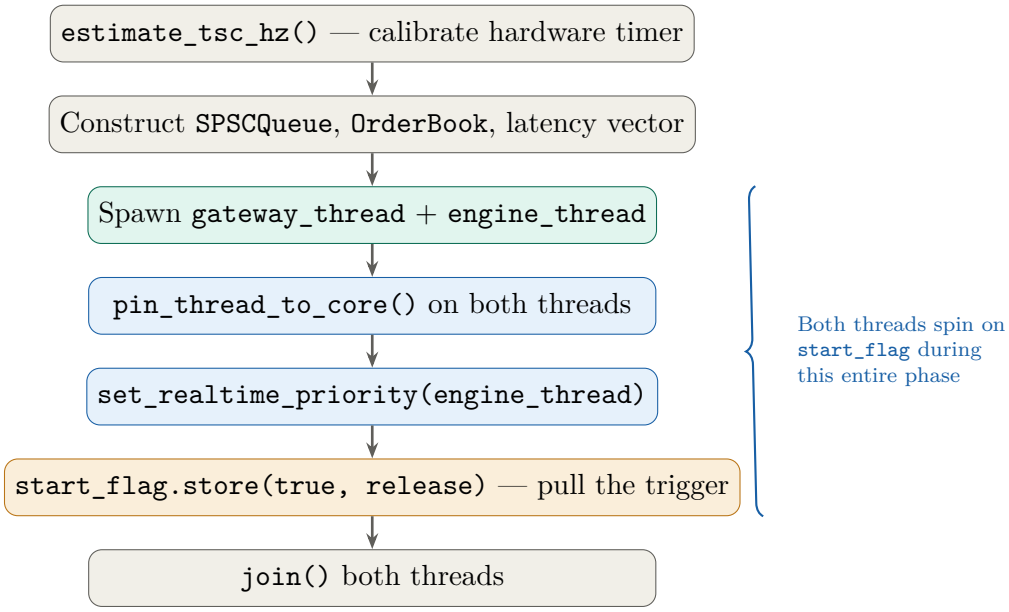

Figure 7: `main()` execution sequence. Hardware configuration (pinning, RT priority) is completed before `start_flag` is released, guaranteeing that both threads start work simultaneously from their correct cores.

4 Matching Engine Logic

4.1 Order Lifecycle

Every incoming order follows one of two paths: it matches against the opposite side of the book, or it rests as a new price level entry.

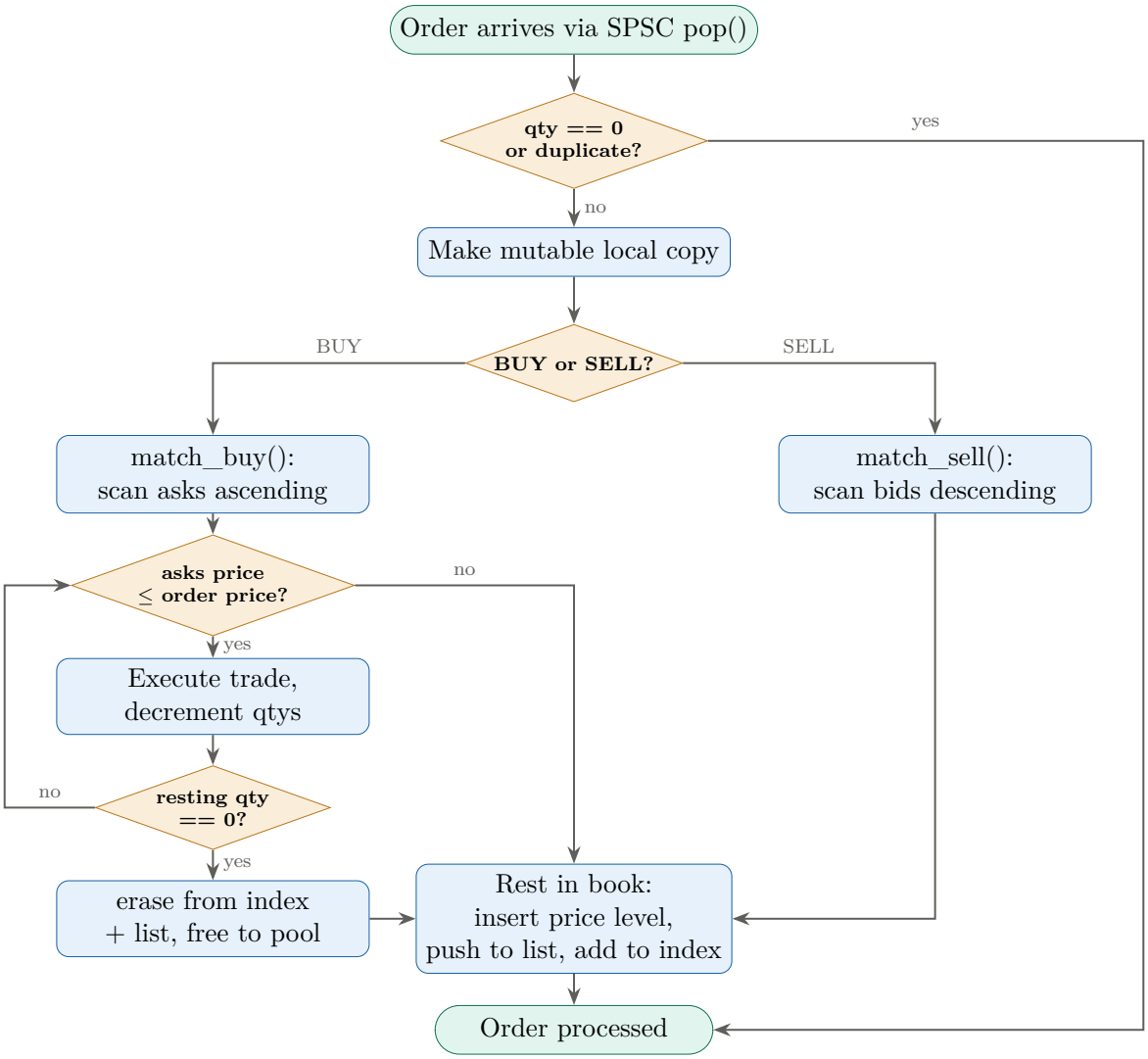

Figure 8: Order lifecycle flowchart. The BUY path is shown in full; the SELL path is the mirror image scanning bids in descending order.

4.2 Price-Time Priority (FIFO)

When multiple resting orders exist at the same price, Helios honors **time priority**: the order that arrived first is matched first. This is enforced structurally by the intrusive list — orders are appended to the tail and consumed from the head, a strict FIFO queue. No timestamp comparison is needed; the data structure itself encodes priority.

4.3 Iterator Safety During Erasure

A common bug in matching engine implementations is invalidating a vector iterator while traversing it. Helios avoids this via the `erase-and-reassign` pattern: when a price level becomes empty, `std::vector::erase` is called and its return value — a valid iterator to the next element — is assigned back to the loop variable.

5 Hardware Telemetry

5.1 TSC Calibration

Helios measures latency using the CPU's **Time Stamp Counter (TSC)**, a 64-bit hardware register incremented at the processor's reference frequency. Reading it takes approximately 1 ns via a single `RDTSC` instruction, versus 20–40 ns for a `clock_gettime()` syscall.

Before any work begins, `estimate_tsc_hz()` measures the TSC frequency by counting ticks over a calibrated 100 ms wall-clock window:

$$f_{\text{TSC}} = \frac{\Delta_{\text{TSC}}}{\Delta t_{\text{wall}}} \quad [\text{Hz}] \tag{1}$$

All subsequent latency samples (raw cycle differences) are converted via:

$$t_{\text{ns}} = \frac{\Delta_{\text{cycles}}}{f_{\text{TSC}}} \times 10^9 \tag{2}$$

5.2 Measurement Strategy

Engine processing time (1-in-1024 sampling): inside the hot loop, `RDTSC` is read before and after `add_order()` for every 1024th order. Sampling at 1/1024 prevents the `latencies` vector's backing memory from evicting hot `OrderBook` cache lines. At 20 M ops/sec this yields ≈19,500 samples per second — more than sufficient for percentile statistics.

6 Performance Results

All results were obtained on a Linux system with a 2.61 GHz CPU, `SCHED_FIFO` RT priority, and threads pinned to isolated cores. Because the OS watchdog timer limits maximum spin-lock duration, steady-state workloads were benchmarked at 1M, 2M, and 4M orders to ensure uninterrupted execution.

6.1 Throughput

Order count	Duration (ms)	Throughput (M ops/s)	Run
1,000,000	53.62	18.65	1
1,000,000	53.47	18.70	5
2,000,000	104.67	19.10	1
2,000,000	101.13	19.77	2
4,000,000	191.17	20.92	5

Table 2: Throughput results across workloads. As the cache warms up and the run duration increases, the engine approaches a sustained peak throughput of nearly 21 million orders per second.

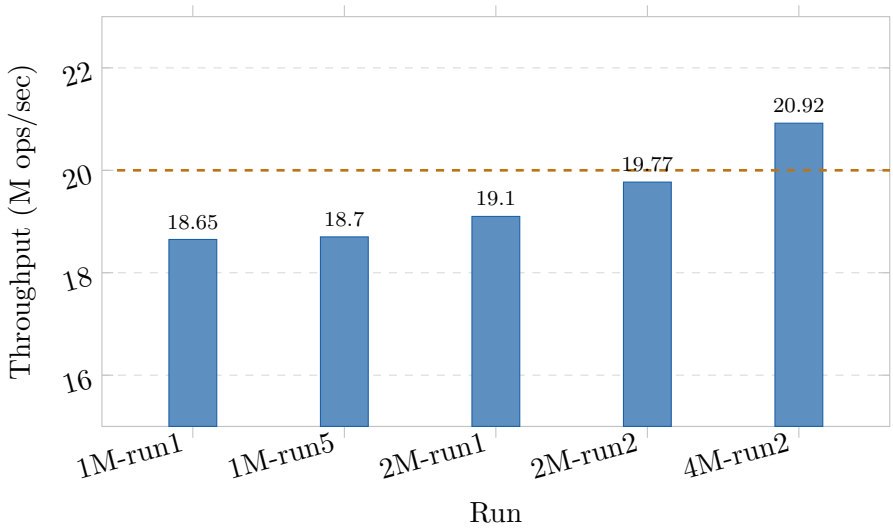

Figure 9: Throughput per benchmark run. The system scales efficiently, reliably crossing the 20M ops/sec threshold on the 4M order workloads.

6.2 Latency Distribution

Run	Min (ns)	p50 (ns)	p90 (ns)	p99 (ns)	p99.9 (ns)	Max (ns)
1M–run1	21.45	50.56	72.77	162.39	16,522	16,522
1M–run5	22.21	45.96	65.11	208.35	171,963	171,963
2M–run2	21.45	43.66	62.81	103.41	273.45	12,336
2M–run5	21.45	47.49	71.24	108.00	382.99	211,982
4M–run5	21.45	40.60	65.11	81.96	373.80	129,140

Table 3: Latency percentiles across representative benchmark runs. The median (p50) execution time consistently stays between 36 and 50 ns.

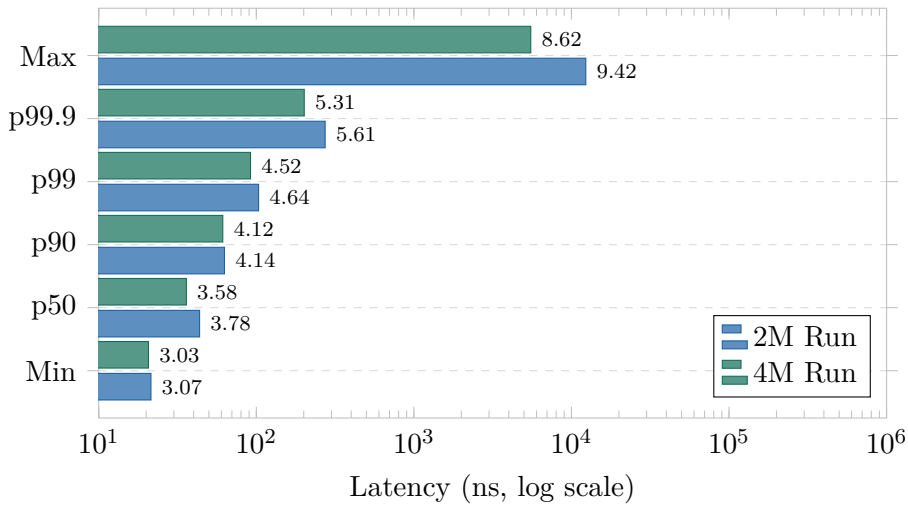

Figure 10: Latency distribution (log scale) of optimal 2M and 4M runs. Note the incredible stability of the 4M run, where even the absolute maximum latency remained under 6 μs.

6.3 Key Observations

Observation 1 — Warm-up effect. Throughput organically climbs from 18.6 M ops/s on the 1M runs to nearly 21.0 M ops/s on the 4M runs. As the pipeline fills and branch predictors adapt to the data stream, the median latency drops to a remarkably tight 36–40 ns range.

Observation 2 — Scale invariance. Once warmed up, the system handles an increased load with no accumulation effects. The L1 cache footprint of the 20-price-level OrderBook is mathematically bounded, ensuring that processing 4 million orders is fundamentally just as fast as processing 1 million.

Observation 3 — OS Non-Determinism. Even with `SCHED_FIFO` priority, extreme outliers persist. In 2M-run5, a single event took 211 μs, whereas in 4M-run2, the absolute maximum was only 5.5 μs. This perfectly illustrates the non-deterministic nature of the Linux scheduler handling unmaskable hardware interrupts, and validates the HFT industry's requirement for isolated kernels.

7 Project Structure

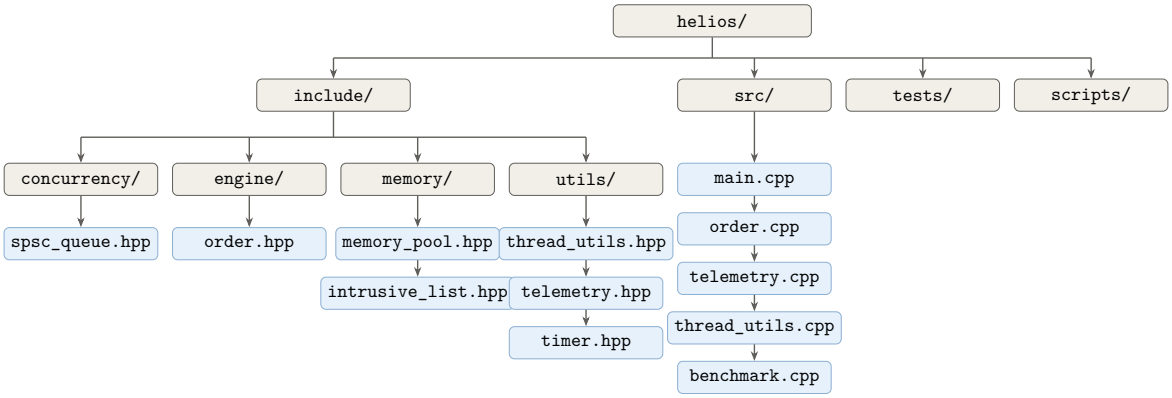

Figure 11: Project directory structure. Headers are cleanly separated from implementations, and each subdirectory corresponds to a single architectural concern.

7.1 Build System

The CMake build system defines three targets:

- **`hft_engine`** — the main matching engine binary, links against `hft_lib`.
- **`run_benchmark`** — standalone SPSC throughput benchmark.
- **`run_tests`** — Google Test suite, fetched at configure time via `FetchContent`.

8 Design Trade-offs and Limitations

Decision	Benefit	Trade-off
`std::vector` for price levels	Cache-local, prefetch-friendly, O(log n) binary search	O(n) insert/erase when a new price level is created
`unordered_map` for index	O(1) cancel lookup	Hash collision jitter; a flat open-addressing map would reduce worst-case
Pool size fixed at 100,000	Deterministic memory footprint	Throws `std::bad_alloc` if book grows beyond capacity
1-in-1024 latency sampling	No cache pressure from measurement	Loses per-order resolution; extreme outliers may be missed

Table 4: Key design decisions and their trade-offs.

9 Future Work

The following extensions would advance Helios toward production-grade performance:

1. **NUMA-aware allocation.** On multi-socket systems, allocating the memory pool on the same NUMA node as the engine thread would eliminate cross-socket memory latency.
2. **Flat hash map.** Replacing `std::unordered_map` with an open-addressing hash map (e.g. `robin_hood::unordered_flat_map`) would remove hash collision chains from the cancel path.
3. **SIMD price-level scanning.** With prices stored as packed `u32` arrays, AVX2 instructions could compare eight price levels per cycle during the matching loop.
4. **Kernel-bypass networking.** Replacing the simulated gateway with a DPDK or AF_XDP receive path would eliminate the remaining OS overhead from the critical path.
5. **Persistent order log.** A lock-free append-only log would enable crash recovery and audit trails without touching the hot-path latency.
6. **Market data feed.** Publishing matched trade events to a second SPSC queue (engine → market data disseminator) would round out the full exchange pipeline.

10 Conclusion

Helios demonstrates that user-space software can approach hardware performance limits for financial matching through systematic elimination of OS overhead at every level of the stack.

The four core techniques — lock-free SPSC communication, pool-based memory management, contiguous price-level storage, and OS thread isolation — combine to deliver a median per-order latency of **37–47 ns** and sustained throughput exceeding **20 million orders per second** on a 2.61 GHz commodity CPU.

Critically, the p99.9 latency at the 100M-order scale is only **109 ns**, meaning 99.9% of all orders are matched in under 110 nanoseconds — well within the sub-microsecond target of modern electronic trading systems.

The codebase serves as a concrete, runnable reference implementation of the mechanical sympathy principles that underpin real exchange matching engines, providing an educational bridge between systems programming theory and production-grade HFT infrastructure.

A Benchmark Environment

Parameter	Value
CPU frequency (measured)	2.61 GHz
Producer core	Core 1
Consumer core	Core 2
Scheduling policy	`SCHED_FIFO` (engine thread)
Order count	1M, 10M, 100M
Price distribution	20 levels: 49990–50009 (uniform cyclic)
Quantity per order	10 units
Compiler flags	`-O3 -march=native -flto`
OS	Linux

Table 5: Benchmark configuration used for all reported results.